Street Fighter Enhanced (Synology)

Browser-Based Street Fighter Game — Cocos2d-HTML5

Technical Foundations and Architecture

Doctum Consilium

Generated 2026-05-08

Abstract

Enhanced browser port of a Street Fighter game built with Cocos2d-HTML5 framework, featuring sprite animation, physics, and Synology NAS deployment. This document covers the theoretical foundations, architectural decisions, and key technical concepts required to understand, contribute to, and maintain this repository.

Contents

Overview

A browser-based 2D fighting game implemented with Cocos2d-HTML5 v3. The game features sprite-sheet animation, collision detection, physics simulation, and a multi-layer canvas rendering architecture. It is containerised with Docker and served as a static web application.

Architecture

```
Browser (Canvas2D / WebGL)
 Cocos2d-HTML5 runtime
     Scene graph (Layer tree)
        BackgroundLayer
        AnimationLayer (fighters)
        StatusLayer (health bars)
     Fighter.js      (state machine)
     FighterPhysics.js (gravity, collision)
     Projectile.js   (hadouken physics)
```

Key Technical Concepts

- Sprite sheet animation: texture atlas (plist) maps named frames to UV coordinates; `cc.AnimationCache` manages frame sequences.
- Scene graph: hierarchical node tree; each node has position, scale, rotation, opacity; parent transforms propagate to children.
- Finite state machine (FSM): fighter states (idle, walking, jumping, attacking, stunned, KO); transitions triggered by input or collision events.
- AABB collision detection: axis-aligned bounding box overlap; $overlap = rect_A.intersects(rect_B)$.
- Game loop: `cc.scheduler` drives `update(dt)` at 60 fps; delta-time normalised physics for frame-rate independence.

Data Flow

Input event (keyboard/touch) → Fighter FSM transition → Physics update (gravity, velocity integration) → Collision detection → Animation frame selection → Canvas render.

Technology Stack

Component	Technology
Game framework	Cocos2d-HTML5 v3
Rendering	Canvas2D / WebGL
Asset format	PNG sprite sheets + plist atlases
Server	nginx (Docker)

Design Decisions

- Cocos2d-HTML5 chosen for its mature sprite/animation API and cross-browser Canvas/WebGL abstraction.
- Static file serving via nginx: no backend required, minimal operational overhead on Synology NAS.

Repository Structure

Listing 1: Top-level directory layout

```
streetfighter-enhanced_syno/
  .github/
    copilot-instructions.md    # GitHub Copilot guardrails
    agents/
      ecr-secrets-agent.agent.md  # ECR secret rotation runbook
  CLAUDE.md                    # Claude Code guardrails
  GEMINI.md                    # Gemini CLI guardrails
  INFRASTRUCTURE.md            # Operational runbook
  ROADMAP.md                   # Execution log and roadmap
  README.md                    # Project overview
```

Contributing

1. Read `README.md`, `INFRASTRUCTURE.md`, and `ROADMAP.md` before any task.
2. Follow the Code Documentation Standard: every public function must have a docstring (Google-style for Python, JSDoc for TypeScript, XML for C#).
3. Run the guardrails validator before committing:

   ```
   git add -A && ./.guardrails/bin/validate_guardrails.sh
   ```

4. For deployment or destructive operations, always use a named tmux session:

   ```
   tmux new-session -A -s deploy-streetfighter-enhanced_syno
   ```